THE PRODUCTION RAG CHEAT SHEET

Fix the 40% Failure Rate: Chunking, Hybrid Search, Reranking, Agentic RAG

7 FAILURE MODES | THE FIXES | RAGAS METRICS | 2026 EDITION

Naive RAG fails to retrieve the right context ~40% of the time - and it's the RETRIEVAL, not the LLM, 73% of the time. This one-page field guide gives AI engineers every failure mode, the fix for each, and the metrics to prove it works. Print it, pin it, ship RAG that's actually grounded.

A free resource from **Tech With Koushik** • **www.koushikmaji.com**

1 THE 7 FAILURE MODES

It's retrieval, not the model

When RAG gives a confident wrong answer, the root cause is usually RETRIEVAL - the system fetched the wrong chunks or none at all. **5 of the 7 failures happen before the LLM sees a single character.** Fix retrieval first; a bigger model can't fix what the retriever never fetched.

#	Failure mode	Where	Symptom
1	Chunking artifacts	Indexing	Fixed-size splits cut sentences/tables; answer is a paragraph away.
2	Vocabulary mismatch	Retrieval	Right meaning, wrong words - 'cancel' vs 'termination policy'.
3	Semantic misses keywords	Retrieval	SKUs, error codes, names get smeared away by embeddings.
4	Arbitrary top-K	Retrieval	K=5 too few for synthesis; K=20 dilutes with noise.
5	No reranking	Retrieval	Best chunk sits at position 7; the model ignores it.
6	Lost in the middle	Context	Right chunk, but buried mid-context = ignored.
7	Stale index / no eval	Index / all	Confident answers from old docs; failures pile up silently.

Key stat: naive RAG fails ~40% of the time; when it fails, it's retrieval ~73% of the time. Meta CRAG: even SOTA RAG answers only ~63% without hallucination.

2 THE FIXES (IN ROI ORDER)

Fix	What to do	Impact
1. Smart chunking	Recursive 512-token default; semantic / parent-child; overlap is a tunable, not a rule.	87% vs 13% accuracy (adaptive vs fixed)
2. Hybrid search	BM25 (keywords) + dense (meaning), fused via Reciprocal Rank Fusion.	Catches exact terms AND concepts
3. Reranking	Cross-encoder re-scores candidates; top-20 -> top-5. HIGHEST-impact fix.	~69% fewer errors (hybrid + rerank)
4. Query transform	Rewrite/expand vague queries; add synonyms; split into sub-queries.	Big recall gains on messy queries
5. Faithfulness check	Verify the answer is grounded before returning it.	Catches hallucinations pre-user

The 2026 production pipeline (7 stages)

```
Parse -> Chunk -> Embed -> Hybrid Retrieve -> Rerank -> Assemble Context -> Faithfulness Check
```

Each stage compensates for the weaknesses of the others. Three-stage naive RAG is a demo, not production.

Agentic RAG & context engineering

Concept	What it means	Payoff / rule
Agentic RAG	Wrap retrieval in a loop: retrieve, grade, reformulate, verify, answer.	+42% faithfulness vs traditional
Cost tradeoff	Agentic ~10x cost + ~5s latency vs naive.	Use only where accuracy pays
Context engineering	Curate WHAT the model sees, not just how you phrase it.	'Context in, prompts out' (Gartner)
Lost-in-the-middle	Models attend to edges, ignore the middle.	Put key info at the edges
Adaptive RAG	Classify each query, route to the right strategy.	The 2026 standard

3 MEASURE IT: RAGAS

RAGAS metric	What it measures	Target
Faithfulness	Is the answer grounded in retrieved context?	> 0.90
Answer relevancy	Does the answer address the question?	> 0.85
Context precision	Are the retrieved chunks relevant?	Higher is better
Context recall	Did you retrieve ALL relevant chunks?	Higher is better
Answer correctness	Is the final answer actually right?	Higher is better

Architecture rule: pick the cheapest architecture that clears your quality bar. Climb Naive -> Advanced -> Agentic -> GraphRAG only when metrics justify the cost & latency.

4 RAG CHECKLIST + QUICK REFERENCE

- ✓ Chunk on meaning (recursive 512 / semantic)
- ✓ Use hybrid search (BM25 + dense + RRF)
- ✓ Rerank with a cross-encoder (top-20 -> 5)
- ✓ Transform/expand vague queries
- ✓ Run a faithfulness check before returning
- ✓ Track RAGAS (faithfulness > 0.9)
- ✓ Refresh the index; kill stale docs
- ✓ Use agentic RAG for complex questions only
- ✓ Route queries (adaptive RAG)
- ✓ Constrain the LLM to retrieved context

2026 STACK
Chunk: recursive 512 / semantic • Embed: text-embedding-3-large, embed-v4, voyage

USE RAG vs FINE-TUNING
RAG: changing knowledge, citations, big corpus. Fine-tune: fixed style/format, stable knowledge, low

ONE-LINE RULES
• Fix retrieval before the model. • Chunk on meaning, not characters. • Always rerank. • Put key info at the edges. • If you don't measure it, it's broken.

RAG isn't dead. Naive RAG is.

More deep-dives & free resources at **www.koushikmaji.com** • Subscribe to **Tech With Koushik**